HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA ĐÀO TẠO SAU ĐẠI HỌC

BÁO CÁO BÀI TẬP LỚN

Đề tài:

AUTOFORMALIZATION PIPELINE SINH VÀ KIỂM CHỨNG HÀNG LOẠT PHÁT BIỂU LEAN 4 TỪ ĐỀ TOÁN NGÔN NGỮ TỰ NHIÊN

Học phần	**:**	**Dữ liệu lớn (INT4418)**
Giảng viên	**:**	**TS. Nguyễn Kiều Linh**
Nhóm	**:**	**05**
Thành viên	**:**	**Lê Quang Đức - B25CHHT086**
		Kim Anh Dũng - B25CHHT087
		Nguyễn Mai Hạnh - B25CHHT093
		Vũ Quốc Hùng - B25CHHT098
		Nguyễn Việt Hưng - B25CHHT099
		Bùi Giang Linh - B25CHHT103

Hà Nội, 2026

MỤC LỤC

TÓM TẮT 4
CHƯƠNG 1. GIỚI THIỆU 5
CHƯƠNG 2. CÔNG TRÌNH LIÊN QUAN 7
CHƯƠNG 3. DỮ LIỆU 8
CHƯƠNG 4. PHƯƠNG PHÁP VÀ KIẾN TRÚC 10
CHƯƠNG 5. THIẾT LẬP THÍ NGHIỆM 13
CHƯƠNG 6. KẾT QUẢ 15
6.1. Chất lượng sinh mã và baseline 15
6.2. Câu hỏi 1: batching và số candidate 16
6.3. Khả năng mở rộng: 100 → 1.000 → 10.000 18
6.4. Câu hỏi 2: phát hiện phát biểu compile được nhưng sai nghĩa 21
6.5. Chịu lỗi và retry 23
CHƯƠNG 7. ABLATION, PHÂN TÍCH LỖI VÀ THREATS TO VALIDITY 25
7.1. Ablation 25
7.2. Phân tích lỗi 26
7.3. Threats to validity 27
7.4. Hạn chế 28
CHƯƠNG 8. KẾT LUẬN, SẢN PHẨM VÀ HƯỚNG PHÁT TRIỂN 29
PHỤ LỤC A. KHAI BÁO SỬ DỤNG AI 30
PHỤ LỤC B. PHÂN VAI VÀ ĐÓNG GÓP 31
PHỤ LỤC C. LỆNH TÁI LẬP 32
PHỤ LỤC D. RUN RECORD VÀ LOG 33
TÀI LIỆU THAM KHẢO 34

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Tiếng Anh	Nghĩa tiếng Việt
AFP	Autoformalization Pipeline	Pipeline tự động hình thức hóa phát biểu toán học
API	Application Programming Interface	Giao diện lập trình ứng dụng
ATP	Automated Theorem Proving	Chứng minh định lý tự động
CPU, vCPU	Central Processing Unit, virtual CPU	Bộ xử lý trung tâm, nhân CPU ảo
GPU	Graphics Processing Unit	Bộ xử lý đồ họa
LLM	Large Language Model	Mô hình ngôn ngữ lớn
NLI	Natural Language Inference	Suy diễn ngôn ngữ tự nhiên
P50, P95	Percentile 50, Percentile 95	Phân vị thứ 50 (median), thứ 95 (đuôi trễ)
Pass@k	Pass at k	Tỷ lệ bài có ít nhất một trong k ứng viên compile được
RAM	Random Access Memory	Bộ nhớ truy cập ngẫu nhiên
REPL	Read-Eval-Print Loop	Vòng lặp đọc – thực thi – in kết quả
SHA-256	Secure Hash Algorithm 256-bit	Hàm băm mật mã 256 bit
T100, T1K, T10K	Tier 100, 1,000, 10,000	Tầng dữ liệu 100, 1.000 và 10.000 bài
UPSERT	Update or Insert	Cập nhật hoặc chèn mới theo khóa
WAL	Write-Ahead Logging	Ghi nhật ký trước khi ghi dữ liệu (SQLite)

TÓM TẮT

Nhóm xây một pipeline sinh hàng loạt phát biểu Lean 4 từ đề toán ngôn ngữ tự nhiên, kiểm chứng chúng bằng Lean, rồi đo nó như một hệ thống dữ liệu lớn ở ba mức 100, 1.000 và 10.000 task. Dữ liệu gồm 22.150 đề từ ProofNet, miniF2F, MATH và GSM8K; mô hình sinh mã là Claude Haiku 5.5; kiểm chứng bằng Lean 4 v4.35.0 và Mathlib. Kiến trúc gồm hai hàng đợi phân vùng, batch inference có cache theo TaskHash, pool Lean worker giữ REPL đã nạp Mathlib, kho kết quả UPSERT và bộ kiểm tra ngữ nghĩa ba lớp. Mỗi lần chạy có run record; mọi bảng và hình sinh bằng script từ log thô.

Bảng 0.1. Tóm tắt các kết quả chính theo câu hỏi nghiên cứu

Câu hỏi	Kết quả chính
Baseline (zero-shot, một bài mỗi request)	Cùng chất lượng với batch job trên 20 bài (20/20 compile, 2/20 bị gắn cờ), nhưng 87.561 so với khoảng 2.000 token mỗi bài
Q1, tầng inference	Gom 20–26 bài mỗi request: token mỗi bài giảm 42 lần, 7,7 % request hỏng; gom 100–142 bài: 58 % request hỏng
Q1, tầng kiểm chứng	Lô 1, 8 hay 32 task cho throughput như nhau ở 2 worker; thời gian nằm ở khởi động REPL và timeout, không ở lớp điều phối
Q1, số candidate	Pass@4 = 1,00 so với Pass@1 = 0,9825, gấp 4 lần token, tỷ lệ bị gắn cờ không đổi (13 %)
Q2	98,9 % compile, nhưng 18,7 % phát biểu compile được bị gắn cờ sai nghĩa; so với người chấm thử: precision 0,73, recall 0,58; có gold thì bắt mọi lỗi cài sẵn
Mở rộng và chịu lỗi	2 worker nhanh gấp 1,74 lần 1 worker trên 2 vCPU; ở 10.000 task, khởi động lại REPL chiếm 45 % thời gian worker; giết một worker vẫn đủ 1.000/1.000 kết quả, 0 dòng trùng

Độ chính xác ngữ nghĩa trên tập audit hiện từ hai LLM chấm thử (81 % và 82 % ĐÚNG); số cuối chờ hai thành viên chấm.

CHƯƠNG 1. GIỚI THIỆU

Đề toán viết bằng ngôn ngữ tự nhiên, khi được dịch sang Lean 4, thì máy kiểm tra được. Nhưng để có hàng nghìn phát biểu như vậy, câu hỏi không còn chỉ là mô hình dịch có giỏi không, mà là hệ thống sinh và kiểm chứng có chịu được tải không, và làm sao biết phát biểu nào đúng nghĩa. Nhóm 5 xây và đo một pipeline như vậy theo bản thiết kế tuần 2 (học phần Dữ liệu lớn INT4418, PTIT, giảng viên TS. Nguyễn Kiều Linh).

Câu hỏi nghiên cứu (hướng dẫn bài tập lớn, Nhóm 5, mục 1):

Q1. Batching và số candidate ảnh hưởng thế nào tới throughput, chi phí và compile rate?

Q2. Làm sao phát hiện trường hợp Lean chấp nhận nhưng phát biểu hình thức sai ý đề tự nhiên?

Nút thắt Big Data: chi phí tính toán và độ trễ. Mỗi Lean worker phải nạp Mathlib (27–45 s để một REPL sẵn sàng, 4,2 GB trang .olean). Thời gian kiểm một phát biểu chênh nhau hàng trăm lần giữa các bài. Mỗi request tới LLM bị giới hạn token đầu ra. Ở hàng nghìn bài, sự kiện hiếm thành chắc chắn. Ba giả thuyết đo được:

H1. Ở 10.000 task, thời gian worker bị chi phối bởi giá của sự kiện hiếm (timeout kéo theo khởi động lại REPL), không phải bởi tốc độ kiểm một phát biểu.

H2. Throughput kiểm chứng tăng theo số Lean worker cho đến số nhân CPU, rồi dừng hoặc giảm.

H3. Gom nhiều bài vào một request LLM giảm token mỗi bài nhưng tăng tỷ lệ request thất bại; ở tầng kiểm chứng, lô lớn giảm chi phí điều phối mỗi task.

Phạm vi. Chỉ formalize phát biểu, phần chứng minh để sorry. Lean 4 v4.35.0, Mathlib v4.35.0, mô hình sinh mã Claude Haiku 5.5. Theo yêu cầu tiết kiệm, chỉ sinh mã cho 1.000 bài; mức 10.000 là trace replay của 1.000 ứng viên thật. Máy 2 vCPU, 7,8 GB RAM, không GPU. Câu hỏi của slide môn học về vai trò Big Data được trả lời ở mục 8.

Đóng góp. (1) Một pipeline phân tán chạy ổn định, có 26 test, log thô và run record cho mỗi lần chạy. (2) Số đo phía hệ thống mà các công trình autoformalization thường

không báo cáo: chi phí kiểm chứng, đuôi trễ, giá của một timeout, giới hạn của song song hóa trên một máy, batching ở cả hai tầng. (3) Bộ kiểm tra ngữ nghĩa ba lớp, mỗi lớp được đo bằng audit và mutation test. (4) Corpus 990 bài, mỗi bài kèm kết luận và cờ ngữ nghĩa.

CHƯƠNG 2. CÔNG TRÌNH LIÊN QUAN

Các công trình gần nhất đo chất lượng mô hình hoặc xây dataset. Phía hệ thống ít khi được báo cáo: throughput, đuôi trễ và chi phí của việc kiểm chứng hàng loạt, cách chịu lỗi, và độ tin cậy của chính bộ kiểm tra ngữ nghĩa.

- Wu và cộng sự, NeurIPS 2022: [1] LLM dịch đúng hoàn toàn 25,3 % đề thi sang Isabelle/HOL; định lý tự formalize nâng tỷ lệ chứng minh trên miniF2F từ 29,6 % lên 35,2 %. Đặt ra bài toán; nhóm làm trên Lean 4, ở quy mô hàng loạt.
- Zheng và cộng sự, 2022 – miniF2F [2] và Azerbayev và cộng sự, 2023 – ProofNet [3]: 488 bài Olympiad và 371 bài đại học có phát biểu hình thức do người viết; nguồn đề và phát biểu gold của nhóm (ProofNet gốc dùng Lean 3, nhóm dùng bản Lean 4).
- Xin và cộng sự, 2024 – DeepSeek-Prover: [4] dịch đề thi sang Lean 4, lọc phát biểu kém, tạo 8 triệu phát biểu có chứng minh, 46,3 % miniF2F với 64 mẫu. Autoformalization quy mô lớn nhưng không báo cáo chi phí kiểm chứng.
- Ying và cộng sự, NeurIPS 2024 – Lean Workbook: [6] 57 nghìn bài, lọc bằng compile và dịch ngược + NLI, 93,5 % đúng trên mẫu kiểm tay. Cùng mục tiêu tách compile với đúng nghĩa.
- Li và cộng sự, NeurIPS 2024 [7]: chọn trong k ứng viên bằng tương đương ký hiệu (ATP) và nhất quán ngữ nghĩa, tăng tương đối 0,22–1,35 lần trên MATH và miniF2F. Gần nhất với lớp kiểm tra bằng Lean của nhóm.
- Lu và cộng sự, ICLR 2025 – FormalAlign [8]: mô hình chấm độ khớp giữa phát biểu tự nhiên và hình thức; một cách thay cho lớp heuristic, nhóm chưa dùng.

Khoảng trống nhóm lấp.

- Nhóm đo pipeline như một hệ thống dữ liệu lớn ở 100 → 10.000 task: chi phí nạp Mathlib, giá của timeout, batching ở cả hai tầng, chịu lỗi.
- Nhóm đo độ tin cậy của bộ kiểm tra ngữ nghĩa bằng audit và mutation test, thay vì chỉ tin vào nó.

CHƯƠNG 3. DỮ LIỆU

Nhóm dùng 22.150 đề thật từ bốn nguồn công khai, đều tải từ GitHub. Dữ liệu chỉ gồm đề toán, không có thông tin cá nhân.

Bảng 3.1. Các nguồn dữ liệu

Nguồn	Số đề	Loại bài	Đáp án	Phát biểu Lean do người viết	Giấy phép	Bản dùng
ProofNet	371	Định lý đại học	Không	Có, bản Lean 4	MIT	DeepSeek-Prover-V1.5 datasets/proofnet.jsonl
miniF2F	488	AMC, AIME, IMO, MATH	Ẩn trong đề	Có, bản Lean 4	MIT / Apache2.0	DeepSeek-Prover-V1.5 datasets/minif2f.jsonl
MATH	12.499	7 môn, mức 1–5	Có	Không	MIT	PRM800K math_splits
GSM8K	8.792	Toán lời văn	Có	Không	MIT	grade-school-math

Thay cho NuminaMath-CoT. HuggingFace không truy cập được từ môi trường thí nghiệm. MATH và GSM8K là hai nguồn con của NuminaMath-CoT và đóng cùng vai trò: nhiều bài có đáp án, không có nhãn hình thức.

ProofNet gốc dùng Lean 3, nên nhóm dùng bản port Lean 4 của DeepSeek-Prover-V1.5; phát biểu gold làm tham chiếu khi kiểm tra tương đương (mục 4). Gold viết cho Mathlib năm 2024: trên v4.35 chỉ 729/859 (84,9 %) còn compile, sau một luật viết lại cú pháp (∑ x in s → ∑ x ∈ s) tăng lên 798/859 (92,9 %); chỉ các gold này được dùng.

Schema và lưu trữ. Mỗi bài là một DataUnit = ⟨problem_id, source, natural_statement, domain, metadata⟩. Bài có đáp án được nối thêm dòng Answer: … vào đề; metadata giữ độ khó, đáp án, gold, độ dài, giấy phép và nguồn gốc. Dữ liệu lưu bằng Parquet theo thư mục Hive source=…/domain=… (3,2 MB), truy vấn trực tiếp bằng DuckDB hoặc Spark.

Split cố định, có phiên bản. Khi chạy, mỗi bài vào partition SHA256(problem_id) mod 16, vì hash() của Python bị salt khác nhau giữa các tiến trình. Ba tầng lồng nhau được lấy mẫu phân tầng theo nguồn với seed 42; thêm T20 (20 bài của T100, cho

baseline một bài mỗi request) và mẫu audit 100 ứng viên. SHA-256 của mọi danh sách nằm trong data/MANIFEST.json.

Bảng 3.2. Số đề theo tầng dữ liệu và nguồn

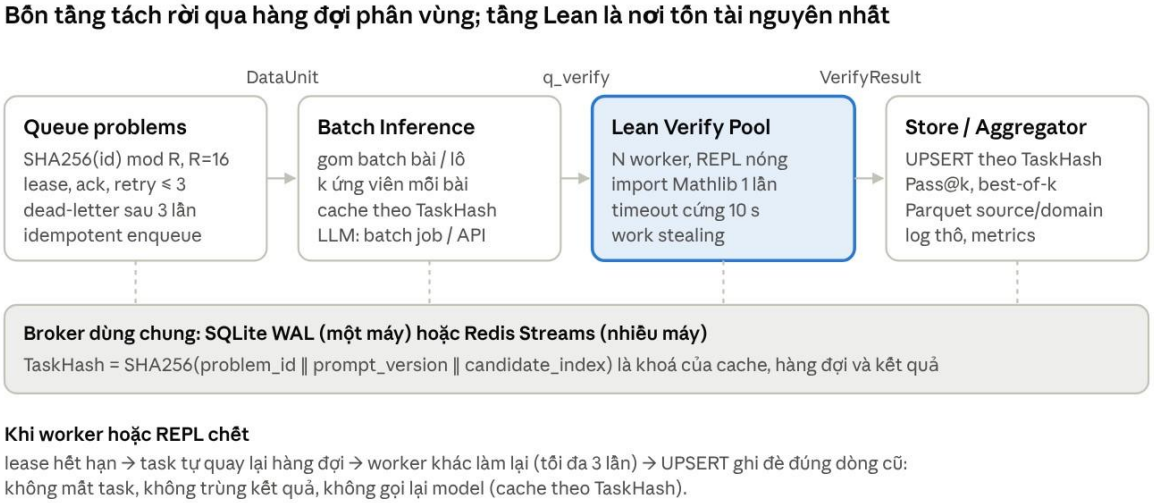
Tầng	ProofNet	miniF2F	MATH	GSM8K	Tổng
T100	4	5	81	10	100
T1K	37	49	814	100	1.000
T10K	371	488	8.141	1.000	10.000

Git không chứa bộ dữ liệu đầy đủ (yêu cầu dữ liệu của đề). Git chỉ giữ data_sample/ (100 đề T100 cùng ứng viên và bản ghi corpus), các split, manifest ghi nguồn, giấy phép, kích thước và SHA-256 từng file, và script tải, dựng lại dữ liệu.

Chất lượng và hạn chế. Loại 1 bản ghi trùng unique_id trong MATH. Phát hiện 2 nhãn MATH sai: math:test/algebra/1730 có nhãn 3 nhưng lời giải ra −3; math:train/intermediate_algebra/1964 có nhãn 9 nhưng tập đúng là {1, 9}. Một số đề GSM8K lỗi định dạng số. Đề công khai từ lâu có thể đã nằm trong dữ liệu huấn luyện của LLM; số token là ước tính (ký tự chia 3,5); so khớp đáp án dạng LaTeX là heuristic. Chi tiết trong docs/DATA_CARD.md.

CHƯƠNG 4. PHƯƠNG PHÁP VÀ KIẾN TRÚC

Pipeline đi đúng luồng của thiết kế tuần 2. Hai hàng đợi phân vùng tách việc sinh mã khỏi việc kiểm chứng, nên mỗi tầng tăng số worker độc lập và một worker hỏng không làm sập cả luồng.



Hình 4.1. Kiến trúc pipeline: bốn tầng tách rời qua hàng đợi phân vùng

Nguồn: src/afp/pipeline.py, queue.py, workers.py, aggregator.py

Đủ bảy thành phần gợi ý của hướng dẫn:

- **Input queue:** hàng đợi bền 16 partition, SQLite WAL cho một máy, Redis Streams cho nhiều máy; lease, ack, nack, release, work stealing (queue.py, redis_backend.py).
- **Prompt/version manager:** prompt có phiên bản v1_zeroshot, v1_zeroshot_1req, v2_fewshot; phiên bản là một phần của TaskHash (prompts.py).
- **Batch LLM inference:** batch job (file vào, file ra) hoặc backend OpenAI-compatible có giới hạn tốc độ và back-off khi gặp lỗi 429 (inference/).
- **Candidate store:** SQLite khóa TaskHash, giữ output thô của mô hình và bản đã qua sanitizer (store.py, data/candidates.db).
- **Lean worker pool:** mỗi worker giữ một Lean REPL đã import Mathlib; maxHeartbeats 400000 và hard timeout (lean/, workers.py).
- **Result/error store:** UPSERT theo TaskHash; trạng thái, loại lỗi, log thô của mỗi lần chạy (store.py, results/raw/runs/).

– **Semantic audit sample:** 100 mẫu đóng băng, phiếu cho hai người chấm, Cohen's κ (audit.py, audit/).

Hợp đồng worker. Mọi task và kết quả mang cùng khóa TaskHash = SHA256(problem_id ‖ prompt_version ‖ candidate_index), và mọi thao tác ghi là UPSERT theo khóa này, nên chạy lại bao nhiêu lần thì kết quả cuối vẫn như nhau. Inference worker nhận ⟨problem_id, natural_statement, prompt_version, k⟩ và trả k ứng viên; Lean verifier nhận ⟨task_hash, lean_code, timeout⟩ và trả ⟨status, error_kinds, latency_ms, peak_ram_mb, worker_id, attempt⟩. COMPILE_OK chỉ được ghi khi Lean thực sự elaborate một khai báo.

Chịu lỗi. Worker lấy việc theo lease 60 s và tự gia hạn khi đang xử lý. Worker hay tiến trình Lean chết thì lease hết hạn và task quay lại hàng đợi; sau 3 lần giao, task vào dead-letter. Trước khi khởi động lại REPL, worker ack các task đã xong và trả phần còn lại của lô cho worker khác (release).

Bộ kiểm tra ngữ nghĩa ba lớp, chỉ áp dụng cho phát biểu đã compile:

Bảng 4.1. Bộ kiểm tra ngữ nghĩa ba lớp

Lớp	Kiểm tra	Bắt được gì
1. Lean, 3–5 lệnh trong REPL nóng, 100.000 heartbeat	giả thiết ⊢ False · giả thiết ⊢ ¬(kết luận) · giả thiết ⊢ kết luận bằng tactic tự động · gold ↔ ứng viên	Giả thiết mâu thuẫn; kết luận sai; định lý ProofNet bị làm tầm thường; lệch nghĩa so với gold. Ngược lại, tương đương gold hoặc đáp án suy ra được từ giả thiết là bằng chứng ủng hộ, chưa phải bảo đảm
2. Heuristic	Đáp án có nằm trong phát biểu không; bài lời văn còn biến không; đáp án âm hoặc phân số mà phát biểu chỉ dùng ℕ	Thiếu đáp án, bài lời văn bị rút thành phép tính, phép trừ hoặc chia bị cắt cụt
3. Người chấm	100 mẫu phân tầng, 3 nhãn, hai người chấm độc lập	Mọi lỗi cần hiểu toán; đo độ tin cậy của hai lớp trên

Kết luận tự động xét theo thứ tự VACUOUS, FALSE, TRIVIAL, ANSWER_MISSING, ARITHMETIC_ONLY (năm cờ sai nghĩa), rồi EQUIV_GOLD, PROVED_CONSISTENT, NO_EVIDENCE_OF_ERROR. Một cờ lỗi được Lean chứng minh là chắc chắn; một xác nhận chỉ cho biết phát biểu tương đương gold hoặc

nhất quán với đáp án, chưa bảo đảm trung thành với đề; chứng minh không được thì chưa nói lên điều gì.

Tổng hợp. Một bài đạt Pass@k nếu có ít nhất một ứng viên COMPILE_OK. Khi có n > k mẫu, trong đó c mẫu compile được, nhóm dùng ước lượng không chệch:

$$Pass@k = 1 - \frac{C(n - c, k)}{C(n, k)}$$

trong đó C(a, b) là tổ hợp chập b của a

Corpus chọn cho mỗi bài một ứng viên: kết luận ngữ nghĩa tốt nhất, rồi mã ngắn nhất, rồi kiểm nhanh nhất. Bài bị gắn cờ vẫn được giữ, kèm cờ.

Độ phức tạp và chi phí (N ứng viên, W worker, P nhân CPU, b bài trong một request LLM):

Bảng 4.2. Độ phức tạp và chi phí của các thành phần

Phần	Chi phí	Số đo của nhóm
Kiểm chứng	T_start + N·t / min(W, P) + n_timeout·(timeout + T_restart)	t = 30–45 ms; T_start = 27–84 s
Lease một lô, không có index	Quét và sắp xếp các task READY, tăng theo backlog	97 ms mỗi lease ở 100.000 task
Lease một lô, có index phủ	Khoảng O(log n + lô)	0,06 ms
Kiểm tra ngữ nghĩa	3–5 lệnh Lean cho mỗi ứng viên	3.121 lệnh cho 989 ứng viên
Token một bài	Khoảng C_request / b + c_bài, b bị chặn trên bởi giới hạn token đầu ra của mỗi request	mục 6.2
Tổng hợp	O(N)	—

CHƯƠNG 5. THIẾT LẬP THÍ NGHIỆM

Baseline bắt buộc (hướng dẫn, mục 7): zero-shot, một bài mỗi request, một candidate, Lean kiểm bằng một worker (cấu hình A: 1 worker, lô 1, timeout 10 s), không repair. Trong môi trường của nhóm, gửi riêng từng bài tốn khoảng 87.600 token mỗi bài, nên baseline sinh mã chạy trên T20 (20 request riêng). Ở ba mức 100, 1.000 và 10.000 bài, ứng viên zeroshot cùng prompt được sinh theo batch job và kiểm bằng đúng cấu hình A. Baseline và phương pháp chính dùng cùng dữ liệu, split, phần cứng và timeout.

Các yếu tố được thay đổi (hướng dẫn, mục 8):

Bảng 5.1. Các yếu tố được thay đổi trong thí nghiệm

Yếu tố	Mức	Giữ cố định	Lần chạy
Quy mô dữ liệu × cấu hình	100 · 1.000 · 10.000 bài × A (1 worker, lô 1) · B (2, 8) · C (4, 32)	timeout 10 s	27
Batch size kiểm chứng	1 · 8 · 32, thêm lô 1 có index hàng đợi	2 worker, timeout 30 s, ba tầng	36
Số Lean worker	1 · 2 · 4 · 8	10.000 task, lô 8, timeout 30 s	6
Số bài mỗi request LLM	1 · 9 · 20–26 · 100–142	cùng prompt	64 request
Số candidate	1 · 2 · 3 · 4	few-shot, 100 bài	400 ứng viên
Zero-shot so với few-shot	v1_zeroshot · v2_fewshot	100 bài	2
Retry	inference: gửi lại phần thiếu; hệ thống: giao task tối đa 1 hoặc 3 lần khi giết worker	1.000 task, 2 worker	6
Ablation	release, heartbeat, index hàng đợi, work stealing, sanitizer	—	mục 7

Metric. Hướng dẫn yêu cầu chọn 1–2 metric chính. Nhóm chọn (1) **độ chính xác ngữ nghĩa trên tập audit** và (2) **throughput ổn định**, tính bằng ứng viên mỗi giây từ lúc worker đầu tiên sẵn sàng đến kết quả cuối. Các chỉ số phụ giải thích trade-off: parse rate, compile rate, Pass@k, P50/P95 thời gian kiểm một ứng viên, throughput tính cả khởi động, token, lỗi theo loại, RAM thật của pool, số timeout và số lần khởi động lại REPL, số task giao lại, dòng trùng và dead-letter.

- **Máy:** 2 vCPU, 7,8 GB RAM, không GPU, máy ảo đám mây. Đợt 16/10: ma trận, đường cong worker, chịu lỗi, broker, ngữ nghĩa. Đợt 26–27/10 (máy khác, cùng số vCPU và RAM, không có swap): baseline, batch size, retry, release, chạy lại ô T1K/A.
- **Phần mềm:** Python 3.13, Lean 4 v4.35.0, Mathlib v4.35.0 (commit 56a0058), REPL v4.35.0; khóa trong environment/.
- **Seed:** các tầng lấy mẫu với seed 42. LLM không đặt được seed, nên ứng viên được cache theo TaskHash và mọi lần chạy dùng lại đúng các ứng viên đó (trace replay).
- **Timeout:** maxHeartbeats 400000 (xác định, không phụ thuộc tải); wall timeout 10 s theo thiết kế tuần 2, hoặc 30 s khi heartbeat là giới hạn chính; lease 60 s, 30 s khi giết worker.
- **Lặp:** 3 lần cho ma trận, batch size, retry, release và ô T1K/A; báo cáo median và min– max. Đường cong worker: 2 lần với 1–2 worker, 1 lần với 4 và 8 worker (12–13 phút mỗi lần).

Run record. Mỗi lần chạy có một file run_record.json theo Phụ lục B của hướng dẫn (Phụ lục D của báo cáo); results/raw/runs_manifest.csv gom record của mọi lần chạy. Các lần chạy ngày 16/10 có trước khi dùng Git; record của chúng được ghi bù, các trường ghi bù được đánh dấu.

CHƯƠNG 6. KẾT QUẢ

6.1. Chất lượng sinh mã và baseline

Zero-shot compile được 98,9 % của 1.000 bài (ProofNet 35/37, miniF2F 49/49, MATH 805/814, GSM8K 100/100). Với mô hình này compile gần như bão hòa, nên compile rate không phân biệt được các cấu hình; chất lượng thật phải đo bằng ngữ nghĩa (mục 6.4). Baseline một bài mỗi request cho chất lượng như batch job trên cùng 20 bài, nhưng tốn gấp 42–50 lần token mỗi bài.

Bảng 6.1. Chất lượng sinh mã và so sánh với baseline

Lần chạy	Ứng viên	Parse rate	Compile rate	Pass@k	Bị gắn cờ ngữ nghĩa	Lean xác nhận đúng	P50 / P95 (ms)
Baseline: 1 bài mỗi request, T20, cấu hình A	20	100 %	100 %	Pass@1 = 1,00	2/20	3/20	35,5 / 99,3
Cùng 20 bài, batch job, cấu hình A	20	100 %	100 %	Pass@1 = 1,00	2/20	4/20	31,0 / 90,8
Zero-shot, T1K	1.000	99,7 %	98,9 %	Pass@1 = 0,989	18,7 %	29,4 %	27,0 / 104,6
Few-shot, T100, 4 mẫu mỗi bài	400	100 %	98,25 %	Pass@1 = 0,9825; Pass@4 = 1,00	12,7 %	28,0 %	21,2 / 82,9

“Bị gắn cờ” và “Lean xác nhận đúng” tính trên số phát biểu compile được. P50/P95 là thời gian Lean kiểm một ứng viên, đo trong cấu hình của từng lần chạy (T1K và T100: 2 worker, lô 8). Nguồn: results/figures/quality_metrics.json, baseline_compare.json.

Baseline so với batch job trên cùng 20 bài. Chất lượng như nhau: 20/20 compile, 2/20 bị gắn cờ “thiếu đáp án” ở cả hai. Chỉ 4/20 phát biểu giống hệt nhau sau khi đổi tên, nên khác biệt giữa hai bên đến từ việc lấy mẫu, không phải từ cách gom. Chi phí thì khác hẳn: 87.561 token và 63 s mỗi bài khi gửi riêng, so với 1.752–2.080 token và 5–10 s mỗi bài khi gom 20– 142 bài (mục 6.2).

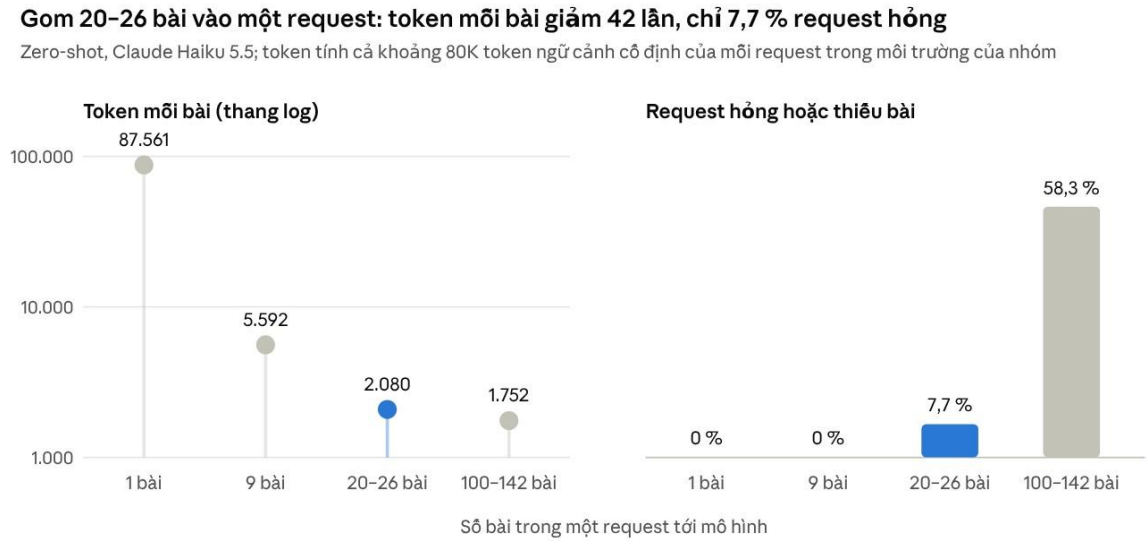
Độ chính xác ngữ nghĩa trên tập audit (metric chính thứ nhất). Trên 100 mẫu audit của T1K, hai người chấm thử bằng LLM chấm ĐÚNG 81 % và 82 % (ĐÚNG hoặc GẦN ĐÚNG: 90 % và 96 %). Đây là chạy thử quy trình. Số liệu cuối phải lấy từ hai thành viên chấm độc lập (audit/rater1.xlsx, audit/rater2.xlsx); python -m afp.study

quality tự cập nhật khi phiếu đã điền. Corpus: 990/1.000 bài có ít nhất một phát biểu compile được, 809 bài trong số đó không bị gắn cờ.

6.2. Câu hỏi 1: batching và số candidate

Batching có hai tầng, và mỗi tầng có một giới hạn riêng. Ở tầng inference, gom bài vào một request chia đều chi phí cố định của request, nhưng request quá lớn chạm giới hạn token đầu ra và hỏng. Ở tầng kiểm chứng, lô lớn giảm chi phí lease cho mỗi task, nhưng chi phí này quá nhỏ để đổi throughput. Thêm candidate tăng Pass@k nhưng không làm phát biểu đúng nghĩa hơn. Compile rate không đổi theo cách gom ở cả hai tầng.

6.2.1. Tầng inference: số bài trong một request



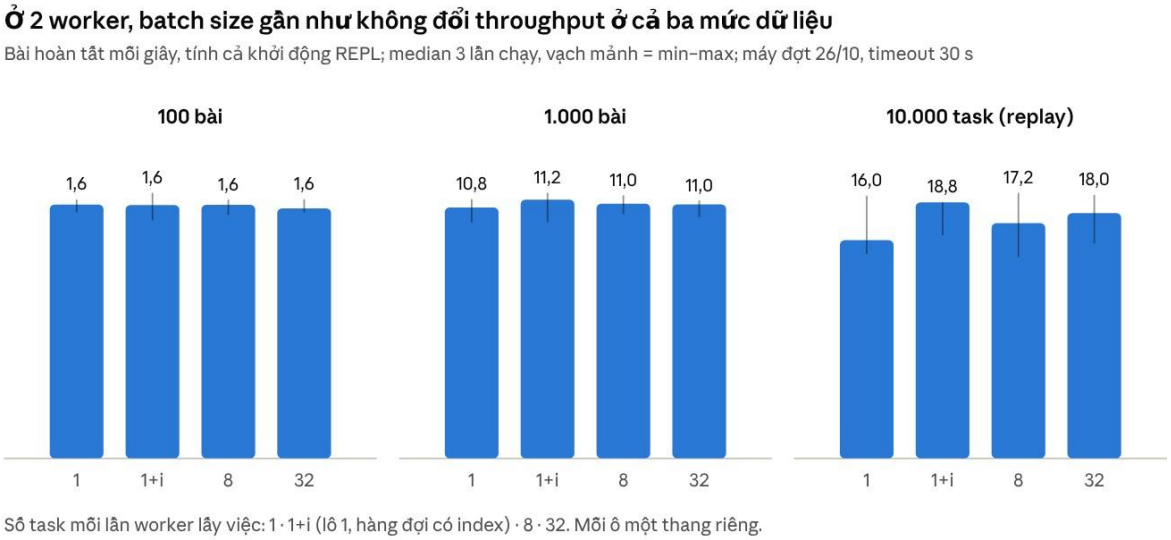
Hình 6.1. Ảnh hưởng của số bài trong một request tới token mỗi bài và tỷ lệ request hỏng
Nguồn: 64 request zero-shot · results/raw/inference_jobs.json, data/jobs/v1_zeroshot_1req/*.manifest.json · results/figures/inference_batching.json

Token mỗi bài gần đúng bằng chi phí cố định của request chia cho số bài, cộng chi phí riêng của mỗi bài (mục 4); giữa 20–26 và 100–142 bài chỉ giảm thêm 16 %. Thời gian mỗi bài: 63 s khi gửi riêng, 10,8 s với 9 bài, 5,2 s với 20–26 bài, 10,0 s với 100–142 bài (tính cả request hỏng).

Request lớn chạm giới hạn 64.000 token đầu ra của mỗi lần trả lời: 7/12 request 100–142 bài hỏng hoặc thiếu bài, so với 2/26 request 20–26 bài và không request 1 hay 9 bài nào. Vì vậy lô phải chia theo ngân sách token đầu ra, không theo số bài (với prompt này khoảng 20–26 bài); H3 đúng ở tầng inference.

6.2.2. Tầng kiểm chứng: số task trong mỗi lần Lean worker lấy việc

Ở 2 worker cố định, batch size gần như không đổi throughput: ở cả ba mức, chênh lệch giữa lô 1, 8 và 32 nhỏ hơn chênh lệch giữa các lần chạy của cùng một cấu hình.



Hình 6.2. Throughput kiểm chứng theo batch size ở ba mức dữ liệu (2 worker)
Nguồn: 36 lần chạy bs_* · results/raw/batch_runs.csv, results/figures/batch_median.csv

– **100 và 1.000 bài:** khoảng 1,6 và 10,8–11,2 bài/s với mọi lô, vì khởi động REPL (khoảng 57 s trên máy đợt 26/10) chiếm phần lớn makespan. Riêng lô 32 ở 100 bài có throughput ổn định thấp hơn (44 so với 58–59 ứng viên/s): 100 task chia thành 4 lô lớn nên cuối lần chạy một worker ngồi không.

– **10.000 task:** median 16,0–18,8 bài/s, nhưng các lần chạy của cùng một lô dao động tới 4,7 bài/s. Mỗi lần chạy gặp 2–6 timeout (ứng viên bệnh lý vượt 30 s), mỗi timeout làm chậm khoảng 1,1 bài/s (hệ số tương quan −0,67): số timeout, không phải batch size, quyết định lần chạy nhanh hay chậm. Lần chạy thứ ba nhanh hơn lần đầu ở mọi lô (khoảng 19 so với 15–16 bài/s); vòng lặp lần chạy đặt ngoài cùng nên độ trôi này chia đều cho các lô.

– **Index hàng đợi:** median 18,8 so với 16,0 bài/s của lô 1, nhưng vẫn trong biên độ của lô 1 (15,0–19,2). Ở backlog 10.000, mỗi lần lease chỉ tốn khoảng 2,5 ms, nhỏ so với khoảng 30 ms một lần kiểm; chỉ khi backlog lên 100.000 task, hàng đợi thiếu index mới thành nghẽn (97 ms so với 0,06 ms mỗi lease, micro-benchmark results/raw/bench_queue.json). Compile rate giống hệt nhau ở mọi lô (99 % ở 100 bài, 98,9 % ở 1.000 và 10.000 task).

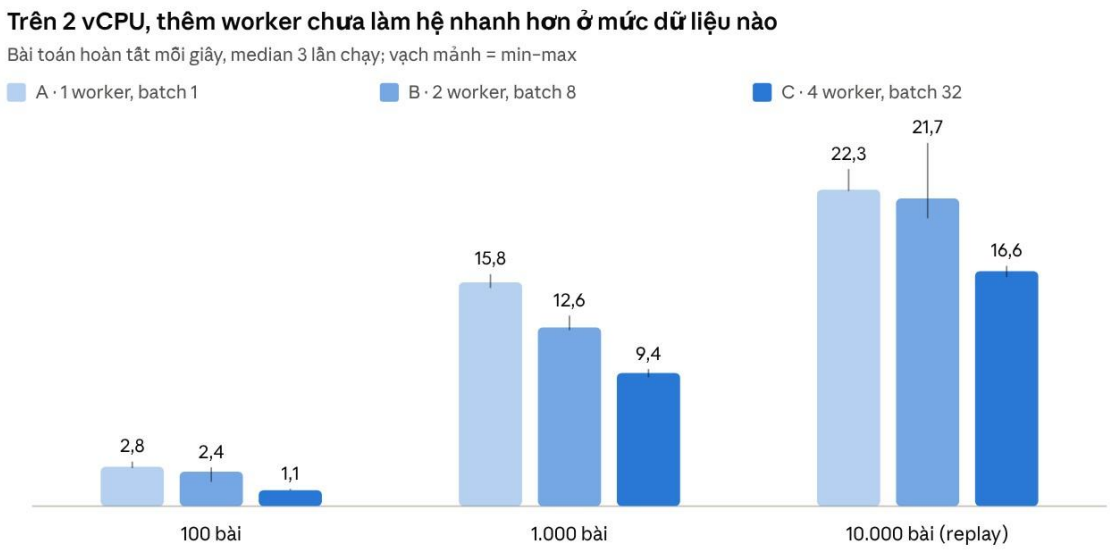
6.2.3. Số candidate và few-shot

- **1 → 4 candidate** (few-shot, 100 bài): Pass@1 = 0,9825, Pass@2 đến Pass@4 = 1,00, đổi lại token tăng tuyến tính (672 → 2.689 token ước tính mỗi bài).
- **Thêm candidate không làm đúng nghĩa hơn.** Bốn ứng viên của cùng một bài giống nhau về ngữ nghĩa: không bài nào có ứng viên bị gắn cờ lẫn ứng viên không bị gắn cờ (0/100), nên chọn best-of-4 vẫn để 13 % bị gắn cờ.
- **Few-shot so với zero-shot** trên cùng 100 bài: compile 100 % so với 99 % ở mẫu đầu, bị gắn cờ 13 % so với 17,2 %, gấp 2 lần token. Ví dụ mẫu giúp ngữ nghĩa (bài GSM8K giữ biến và giả thiết) hơn là giúp compile.
- **Retry ở tầng inference** là bắt buộc khi gom lớn: không gửi lại phần thiếu thì chỉ có 784/1.400 ứng viên (56 %); gửi lại theo TaskHash trong các request 9–26 bài thì đủ 1.400/1.400.

6.3. Khả năng mở rộng: 100 → 1.000 → 10.000

Trên máy 2 vCPU, thêm worker theo thiết kế tuần 2 không làm hệ nhanh hơn ở cả ba mức dữ liệu: ở 100 và 1.000 bài vì chi phí khởi động REPL, ở 10.000 task vì mỗi timeout kéo theo khởi động lại REPL, chiếm gần nửa thời gian worker (giả thuyết H1 đúng). Khi heartbeat là giới hạn chính, 2 worker nhanh gấp 1,74 lần 1 worker; vượt số nhân thì throughput sụp (H2 đúng).

Ma trận 3 mức dữ liệu × 3 cấu hình × 3 lần, wall timeout 10 s và maxHeartbeats 400000 như thiết kế tuần 2. Vòng lặp lần chạy nằm ngoài cùng, để trôi hiệu năng của máy chia đều cho các ô.



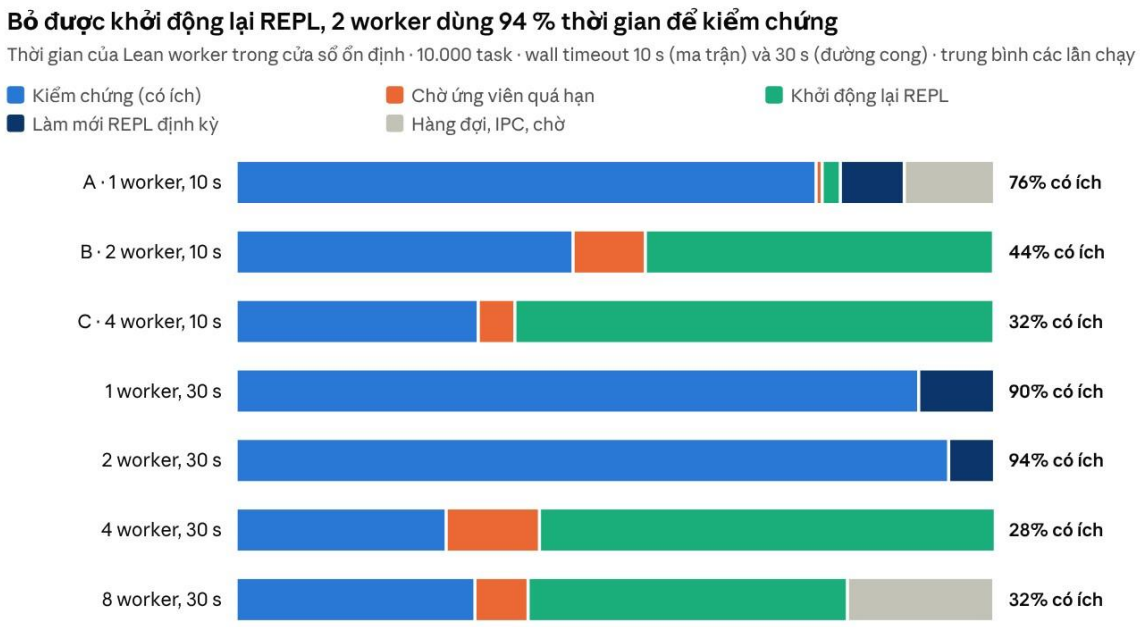
Hình 6.3. Throughput theo quy mô dữ liệu và cấu hình worker (ma trận 3 × 3)
Nguồn: 27 lần chạy mx_* · results/raw/matrix_runs.csv, results/figures/matrix_median.csv

Ma trận dùng bộ ứng viên sanitizer v1 để workload cố định; compile rate giống hệt nhau ở mọi ô cùng mức (98,0 % ở 100 bài, 98,3 % ở 1.000 và 10.000), tức song song hóa không đổi kết quả kiểm chứng. Bảng đầy đủ (P50/P95/P99, khởi động, RAM, timeout từng lần): results/figures/matrix_median.csv.

– **100 và 1.000 bài:** A nhanh nhất. Khởi động REPL mất 27–31 s với 1 worker, 37–38 s với 2 và 80–83 s với 4, vì các REPL cùng nạp Mathlib trên 2 vCPU. Ở 1.000 bài, B và C mỗi lần chạy còn gặp 1 timeout và mất khoảng 40 s khởi động lại.

– **10.000 task:** A và B ngang nhau, C chậm hơn 26 %. 10 ứng viên bệnh lý (0,1 %) làm B timeout 7–9 lần và C 10 lần mỗi lần chạy; A chạy một mình nên các ứng viên đó gần như luôn dừng bằng heartbeat sau khoảng 4 s (1 timeout trong 3 lần chạy). P95 tăng theo số worker (68 → 87 → 183 ms) vì 2 vCPU bị chia.

– **RAM:** footprint thật của pool khoảng 5 GB dù 1, 2 hay 4 worker, vì 4,2 GB trang .olean được ánh xạ dùng chung; mỗi REPL chỉ thêm khoảng 0,35 GB heap riêng (ΣRSS đếm trùng trang chung nên báo tới 18 GB).

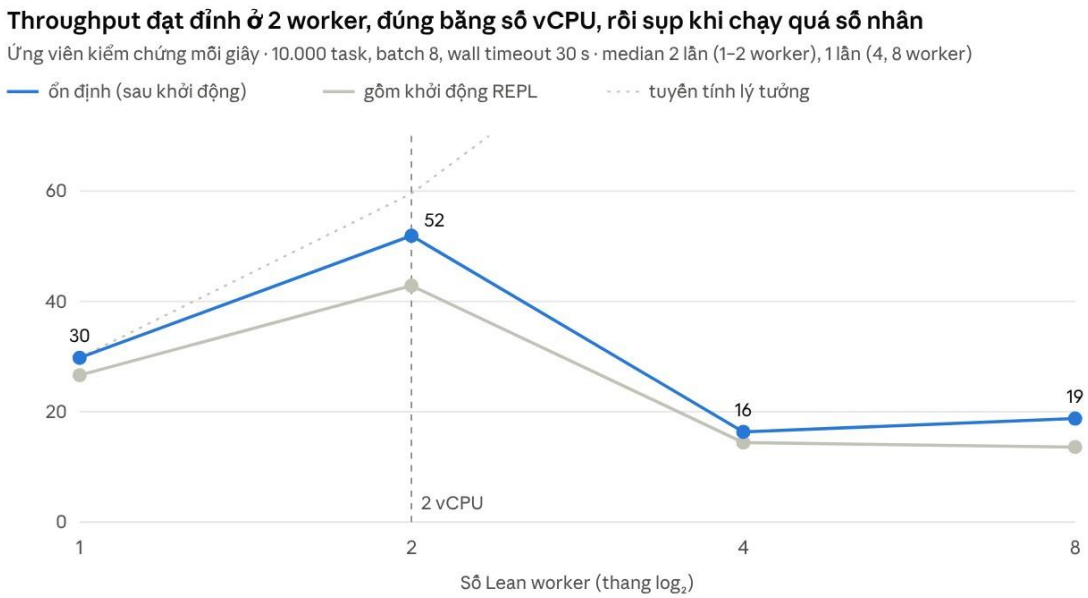


Hình 6.4. Cơ cấu thời gian của Lean worker ở 10.000 task

Nguồn: 9 lần chạy mx_T10K_* và 6 lần sc_T10K_* · results/figures/time_budget_*.json

H1. Với cấu hình B ở 10.000 task (median 3 lần), khởi động lại REPL chiếm 45 % thời gian worker và chờ timeout thêm 10 %; kiểm chứng thật chỉ còn 46 %. Với C, khởi động lại chiếm 63 %. Một timeout 10 s thực tế kéo theo 30–200 s khởi động lại, vì REPL mới phải nạp lại Mathlib trong lúc các worker khác vẫn chiếm CPU.

Đường cong worker (10.000 task, lô 8, wall timeout 30 s để heartbeat là giới hạn chính; 1–2 worker chạy 2 lần, 4 và 8 worker 1 lần vì mỗi lần mất 12–13 phút):



Hình 6.5. Đường cong throughput theo số Lean worker

Nguồn: 6 lần chạy sc_T10K_* · results/raw/scaling_runs.csv, results/figures/scaling_median.csv

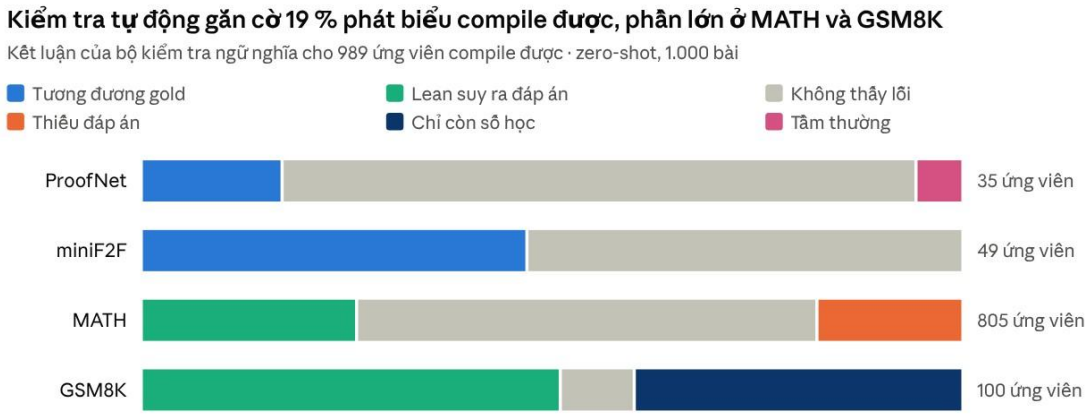
Từ 1 lên 2 worker, throughput ổn định tăng từ 29,8 lên 51,9 ứng viên/s (1,74 lần); Lean bận 90–94 % thời gian và không còn timeout nào. Với 4 và 8 worker, throughput sụp xuống 16,3 và 18,8 ứng viên/s: chạy quá số nhân làm cả 10 ứng viên bệnh lý vượt 30 s, mỗi lần khởi động lại REPL mất 57–217 s với 4 worker và 90–300 s với 8 worker (2 REPL không khởi động lại được). Khi chạy quá số nhân, 4,2 GB trang .olean dùng chung bị đẩy dần khỏi page cache (trang thường trú, median: 4,4 GB với 1 worker, 2,8 GB với 4, 1,9 GB với 8, kèm tới 3 GB swap): đây là lúc giả thuyết "tràn RAM" của tuần 2 thành đúng.

Ước lượng năng lực. Mỗi nhân kiểm khoảng 26 ứng viên/s khi không chạy quá số nhân. Kiểm 1 triệu ứng viên trong 1 giờ cần khoảng 278 ứng viên/s, tức khoảng 11 nhân: 6 máy 2 vCPU, 8 GB, mỗi máy 2 worker, chung một Redis. Đây là ngoại suy từ một máy; worker ở tiến trình khác đã chạy được (mục 6.5), nhưng chưa đo trên nhiều máy thật.

6.4. Câu hỏi 2: phát hiện phát biểu compile được nhưng sai nghĩa

Lean chỉ bảo đảm phát biểu đúng kiểu. Nhóm phát hiện sai nghĩa bằng ba lớp chồng nhau và đo từng lớp. Kiểm tra tự động gắn cờ 18,7 % phát biểu compile được và xác nhận đúng 29,4 %. So với người chấm thử, precision 0,73 và recall 0,58. Mutation test cho thấy: khi có phát biểu gold, mọi lỗi cài sẵn đều bị bắt; khi không có gold, đáp án sai chỉ bị bắt 26 %.

Kiểm tra tự động trên 989 ứng viên zero-shot compile được (3.121 lệnh Lean, khoảng 26 phút với 2 worker):



Hình 6.6. Kết quả kiểm tra ngữ nghĩa tự động trên 989 ứng viên compile được
Nguồn: results/raw/runs/q_T1K_zeroshot/semantic.jsonl · results/figures/semantic_summary.json

– **185 phát biểu (18,7 %) bị gắn cờ:** 143 thiếu đáp án (MATH), 40 chỉ còn là phép tính (GSM8K), 2 tầm thường (ProofNet). Không ứng viên nào bị Lean chứng minh là có giả thiết mâu thuẫn hay bị phản chứng.
– **Bằng chứng tích cực:** 29 phát biểu được Lean chứng minh tương đương với gold (23/49 miniF2F, 6/35 ProofNet); với 262 bài có đáp án, Lean tự suy ra đáp án từ giả thiết. 513 phát biểu "không thấy lỗi" là vùng kiểm tra tự động không kết luận được, phải nhờ người chấm. Xác nhận của Lean cũng chưa đủ: trong 29 mẫu audit được xác nhận, 27 được cả hai người chấm thử chấm ĐÚNG; mẫu math:train/prealgebra/251 chỉ còn Nat.factorial 4 = 24, chứng minh được nhưng mất nội dung đề.
– **Few-shot ít bị gắn cờ hơn** trên cùng 100 bài: 13 % so với 17,2 % (mẫu đầu tiên của mỗi bài). Khác biệt nằm ở GSM8K: 4/10 phát biểu zero-shot chỉ còn phép tính, 0/40 với fewshot, vì ví dụ mẫu formalize bài lời văn bằng biến và giả thiết.

Audit 100 mẫu. Mẫu phân tầng 100 ứng viên compile được của T1K (ProofNet 20, miniF2F 20, MATH 45, GSM8K 15), chấm 3 nhãn ĐÚNG / GẦN ĐÚNG / SAI theo docs/AUDIT_GUIDELINE.md; phiếu cho hai thành viên ở audit/rater1.xlsx, audit/rater2.xlsx. Để thử quy trình, hai LLM (Claude Sonnet, Claude Opus) chấm độc lập trước, không thay cho audit người: đồng thuận 87 % trên 3 nhãn (Cohen's $\kappa = 0{,}59$), $\kappa = 0{,}83$ cho ĐÚNG so với phần còn lại nhưng chỉ 0,39 cho SAI. 7/13 bất đồng là bài GSM8K bị formalize thành đẳng thức số học trần như 14 - (2 + 4) = 8; hướng dẫn chấm v2 đã chốt trường hợp này là SAI.

Từng lớp so với người chấm thử (dương tính = không được chấm ĐÚNG; results/figures/semantic_layers.json):

Bảng 6.2. Độ chính xác và độ phủ của từng lớp kiểm tra so với người chấm thử

Lớp	Gắn cờ trong 100 mẫu	So với người chấm A: precision / recall	So với người chấm B: precision / recall
Chỉ Lean	2	0,50 / 0,05	0,00 / 0,00
Chỉ heuristic	13	0,77 / 0,53	0,77 / 0,56
Cả hai lớp	15	0,73 / 0,58	0,67 / 0,56

Kiểm tra tự động bắt tốt lỗi có dấu hiệu hình thức nhưng bỏ sót lỗi cần hiểu toán: cả 4 mẫu người chấm B chấm SAI đều lọt. Ví dụ math:test/algebra/2167 vô nghĩa trên

$\mathbb{R}$ vì không có a, b thực nào thỏa giả thiết; proofnet:Dummit-Foote:exercise_4_2_8 đúng tầm thường vì Lean quy ước Subgroup.index = 0 cho chỉ số vô hạn; math:train/counting_and_probability/201 thay bài xác suất bằng một đẳng thức nhị thức.

Mutation test. Lấy 100 phát biểu gold compile được (miniF2F, ProofNet), cố ý chèn một loại lỗi, giữ các bản vẫn compile (483 bản, 2.415 lệnh Lean), rồi chạy đúng bộ kiểm tra ở trên.

Khi có gold, mọi bản lỗi đều mất tương đương với gold và bị phát hiện; chỉ 2/100 bản đối chứng bị báo nhầm. Khi không có gold (trường hợp của MATH và GSM8K), chỉ lỗi cấu trúc bị bắt chắc chắn (giả thiết mâu thuẫn 99 %, kết luận tầm thường 100 %); đáp án sai chỉ bị Lean phản chứng 26 %, quan hệ bị đảo 15 %, thiếu giả thiết 0 %.

6.5. Chịu lỗi và retry

Lease cộng UPSERT theo TaskHash cho kết quả đúng một lần khi worker chết. Retry là phần bắt buộc: không giao lại task thì mỗi lần giết worker mất 0,5 % kết quả; có giao lại thì đủ 1.000/1.000, đổi lại makespan tăng khoảng 13 %.

Bảng 6.3. Kết quả thí nghiệm chịu lỗi và retry

Thí nghiệm (1.000 task, 2 worker, lô 8)	Kết quả
Giết lean-0 sau 302 kết quả, lease 30 s	1.000/1.000 kết quả; 8 task giao lại, 3 kết quả ghi đè đúng dòng cũ; 0 dòng trùng; 0 lần gọi lại mô hình
Retry: giao tối đa 3 lần, 3 lần chạy	1.000/1.000 cả 3 lần; 0 dead-letter; 0 dòng trùng; makespan median 99,7 s (98,0–104,6)
Không retry: giao tối đa 1 lần, 3 lần chạy	995/1.000 cả 3 lần; 8 task vào dead-letter; makespan median 88,0 s (86,6–89,8)
Retry ở tầng inference (1.400 ứng viên)	Không gửi lại: 784/1.400 (56 %); gửi lại phần thiếu theo TaskHash trong các job 9–26 bài: 1.400/1.400

Vì sao 995 chứ không phải 992: worker ghi kết quả bằng UPSERT ngay sau khi kiểm nhưng ack cả lô một lần, nên 3 task đã có kết quả vẫn vào dead-letter; khi có retry, chính 3 task này được kiểm lại và ghi đè đúng dòng cũ. Ngoài ra: sự cố thật ở 8 worker làm 2 worker chết, 6 worker còn lại vẫn hoàn tất đủ 10.000 kết quả; cùng 1.000 task trên broker SQLite và Redis cho 29,0 và 27,5 ứng viên/s, và một worker tham gia sau từ tiến

trình độc lập (chỉ biết URL Redis) xử lý 376/1.000 task, 0 dòng trùng; 9 lần chạy T1K cho trạng thái giống hệt nhau trong mỗi cấu hình, khác nhau giữa các cấu hình ở đúng 1/1.000 bài (mục 7.2). Nguồn: results/figures/fault_tolerance.json, retry_ablation.json, broker_comparison.json, determinism.json.

CHƯƠNG 7. ABLATION, PHÂN TÍCH LỖI VÀ THREATS TO VALIDITY

7.1. Ablation

Mỗi thành phần chính được tắt hoặc thay một lần, giữ nguyên phần còn lại. Bảng sinh bằng python -m afp.ablation; mỗi ô đọc từ file nguồn ghi trong results/figures/ablation.json.

Bảng 7.1. Kết quả ablation các thành phần chính

Thành phần (thiết lập)	Có	Không có
REPL nóng: nạp Mathlib một lần mỗi worker (kiểm một ứng viên)	45,8 ms	20,2 s (nhanh hơn 442 lần)
Heartbeat là giới hạn chính, wall timeout 30 s thay vì 10 s (10.000 task, 2 worker)	51,9 ứng viên/s, 0 timeout	24,2 ứng viên/s, 7 timeout
Trả phần còn lại của lô về hàng đợi trước khi khởi động lại REPL (1.000 task, 2 worker, timeout 10 s, 3 lần mỗi bên)	ổn định 43,2 s, makespan 106,2 s	ổn định 84,9 s, makespan 147,4 s
Index phủ cho hàng đợi (lease khi còn 100.000 task; đầu cuối 10.000 task)	0,06 ms/lease; T10K: 18,8 bài/s (median)	96,8 ms/lease; T10K: 16,0 bài/s (median; 15,0–19,2)
Work stealing thay cho chia tĩnh theo partition (mô phỏng trên 1.000 latency thật, 8 worker)	10 s (lý tưởng 5,9 s)	chia tĩnh: 14,7 s
Sanitizer v2, giữ:= của let / have (1.000 output thô zero-shot)	compile 98,9 %, 3 lỗi cú pháp	compile 98,3 %, 10 lỗi cú pháp

Hai nhóm thành phần quyết định thời gian: REPL nóng (442 lần), và các cơ chế giảm giá của sự kiện hiếm: để heartbeat làm giới hạn chính tăng throughput 2,1 lần, trả lô về hàng đợi giảm makespan 28 % vì worker kia làm nốt lô thay vì chờ REPL mới nạp lại Mathlib. Work stealing đã có lợi từ 2 worker (mô phỏng: 23,8 so với 29,8 s nếu chia tĩnh) và càng quan trọng khi nhiều worker; index hàng đợi chỉ cần khi backlog lên khoảng 100.000 task. Các so sánh bật/tắt còn lại (gom bài mỗi request, few-shot, số

candidate, retry hai tầng, từng lớp kiểm tra ngữ nghĩa, so với gold) đã có ở mục 6; bảng đủ 13 dòng: results/figures/ablation.md.

7.2. Phân tích lỗi

Ở lần chạy đầu, 17 trong 1.000 ứng viên zero-shot không compile. Truy nguyên nhân gốc cho thấy chỉ 8 lỗi (0,8 %) thuộc về mô hình; 7 lỗi do chính sanitizer của nhóm, 1 do header cố định, 1 do giới hạn thời gian (results/figures/failure_causes_*.json).

Bảng 7.2. Nguyên nhân gốc của các ứng viên không compile

Nguyên nhân gốc	Số	Ví dụ và cách xử lý
Sanitizer v1 cắt nhầm let …:=	7	proofnet:Munkres:exercise_13_6: “unexpected end of input”; sanitizer v2 bỏ qua:= thuộc let / have, chạy lại trên output thô đã lưu
Kiểu, coercion, instance	4	Finset.Icc 1 100 bị suy ra kiểu ℝ (“failed to synthesize LocallyFiniteOrder ℝ “)
Cú pháp Lean 4	2	Trộn binder ∃ j (hj: …); thiếu ngoặc đóng
Thiếu namespace hoặc notation	2	$*_v$ cần open Matrix; IsTopologicalBasis cần tiền tố TopologicalSpace.
Quá hạn 10 s	1	Bài 933: tìm instance DecidablePred cho lượng từ lồng nhau
Header mở Nat	1	open Nat biến φ thành ký hiệu hàm Euler, mà mô hình dùng φ làm tên biến

Taxonomy bắt được lỗi của chính pipeline. Vì output thô được lưu nguyên, sửa sanitizer chỉ cần chạy lại bước hậu xử lý: 8/1.400 ứng viên thay đổi, không gọi lại mô hình lần nào; 6/7 ứng viên bị sanitizer v1 cắt nhầm nay compile được, ứng viên còn lại lộ ra một lỗi thật của mô hình. ProofNet compile thấp nhất; cả 2 lỗi còn lại đều là bài Topology của Munkres.

Trạng thái phụ thuộc tải. Ứng viên bài 933 chạy một mình hết heartbeat sau 4,4 s và trả TYPE_MISMATCH; chạy cùng worker khác thì vượt 10 s và thành TIMEOUT. Heartbeat là giới hạn xác định, đồng hồ thì không, nên wall timeout chỉ nên là lưới an toàn rộng hơn nhiều.

7.3. Threats to validity

- **Nội tại – một mô hình sinh mã, không đặt được seed.** Chênh lệch nhỏ giữa hai cách sinh trên 20 bài không có ý nghĩa thống kê; mọi thí nghiệm hệ thống dùng cùng bộ ứng viên cache theo TaskHash.
- **Nội tại – token gồm khoảng 80K ngữ cảnh cố định mỗi request** của môi trường chạy mô hình: phóng đại chênh lệch giữa gửi riêng và gom so với gọi API trực tiếp; kết luận chỉ dựa vào chiều và bậc độ lớn.
- **Nội tại – mức 10.000 là trace replay 10 bản của 1.000 ứng viên:** 10 ứng viên bệnh lý là một ứng viên nhân 10; tỷ lệ 0,1 % giữ đúng như ở 1.000 bài, Lean vẫn kiểm thật đủ 10.000 task.
- **Nội tại – hai đợt máy khác nhau.** Máy đợt 26–27/10 không có swap; chạy lại ô T1K/A ba lần trên đó cho 8,6 bài/s (7,8–11,6) so với 15,8 ở ma trận. Thời gian Lean của các ứng viên không timeout gần như bằng nhau (32,0 so với 34,0 s); khác biệt đến từ khởi động REPL lâu hơn (40 so với 27 s) và một timeout 10 s ở 2/3 lần chạy (results/figures/repro_T1K_A.json). Vì vậy báo cáo chỉ so sánh các lần chạy trong cùng một đợt.
- **Cấu trúc – độ chính xác ngữ nghĩa và κ hiện từ hai LLM chấm thử** cùng họ với mô hình sinh mã: có thể cùng thiên lệch, làm số đo cao hơn thực tế; không dùng làm số cuối.
- **Cấu trúc – heuristic so khớp đáp án LaTeX quá chặt,** gây báo nhầm "thiếu đáp án"; precision và recall từng lớp đã đo (mục 6.4).
- **Thống kê – 100 mẫu audit; 4 và 8 worker chạy 1 lần.** 81 % trên 100 mẫu có khoảng Wilson 95 % là 72–88 %; kết luận ở 4 và 8 worker là định tính (sụp).
- **Bên ngoài – một máy 2 vCPU; đề công khai có thể đã nằm trong dữ liệu huấn luyện; NuminaMath thay bằng MATH và GSM8K.** Đỉnh của đường cong worker sẽ khác trên máy nhiều nhân; không diễn giải compile rate như năng lực tổng quát của mô hình.

7.4. Hạn chế

- Chỉ sinh mã cho 1.000 bài; mức 10.000 là trace replay. Mô hình chạy dạng batch job, nên chưa đo được độ trễ từng request của tầng inference; backend OpenAI-compatible đã có và đã test với server giả lập.
- Chưa có sandbox dạng container: mỗi REPL là một tiến trình riêng, bị giới hạn bằng heartbeat và wall timeout, nhưng không bị giới hạn bộ nhớ hay mạng.
- Nhiều máy dùng Redis Streams, đã chạy nhưng chưa đo trên nhiều máy thật; kiểm tra ngữ nghĩa tự động chỉ chứng minh được một chiều.

CHƯƠNG 8. KẾT LUẬN, SẢN PHẨM VÀ HƯỚNG PHÁT TRIỂN

Trả lời Q1. Batching có lợi lớn ở tầng inference và gần như trung tính ở tầng kiểm chứng. Gom 20–26 bài mỗi request giảm token mỗi bài 42 lần; gom hơn 100 bài thì 58 % request hỏng, nên lô phải chia theo ngân sách token đầu ra. Ở tầng kiểm chứng, lô 1, 8 hay 32 task cho throughput như nhau vì thời gian nằm ở khởi động REPL và timeout, không ở lease. Thêm candidate đưa Pass@4 lên 1,00 với gấp 4 lần token mà không giảm tỷ lệ bị gắn cờ; compile rate không đổi theo cách gom.

Trả lời Q2. Dùng ba lớp có đo độ tin cậy: Lean (tương đương với gold, phản chứng, giả thiết mâu thuẫn), heuristic về đáp án và cấu trúc, và người chấm trên mẫu phân tầng. Có gold thì kiểm tra tự động bắt mọi lỗi cài sẵn (2 % báo nhầm); so với người chấm thử, nó chỉ bắt 58 % số phát biểu không được chấm ĐÚNG (67 % ở bài chỉ có đáp án) và bỏ sót lỗi cần hiểu toán. Vì vậy audit người là bắt buộc, corpus giữ kết luận và cờ cho từng bài, và báo cáo không tuyên bố đúng nghĩa chỉ dựa vào compile.

Giả thuyết (mục 1). H1 đúng: khởi động lại REPL chiếm 45 % thời gian worker ở 10.000 task với 2 worker, 63 % với 4 (mục 6.3). H2 đúng: 2 worker nhanh gấp 1,74 lần 1 worker trên 2 vCPU, còn 4 và 8 worker chậm hơn cả 1 worker. H3 đúng nhưng ở tầng kiểm chứng lợi ích không đáng kể: token mỗi bài giảm 42 lần và tỷ lệ request hỏng tăng theo cỡ lô; lô lớn giảm chi phí lease mỗi task (2,5 → 0,1 ms ở backlog 10.000) nhưng lô 1, 8, 32 cho throughput như nhau (mục 6.2).

Vai trò Big Data (câu hỏi trên slide môn học) được chứng minh bằng các thí nghiệm có kết luận đổi khi dữ liệu tăng: worker phải giữ trạng thái (REPL nóng 45,8 ms so với 20,2 s khi chạy lạnh); 0,1 % ứng viên bệnh lý gây 7–10 lần khởi động lại REPL mỗi lần chạy; song song hóa dừng ở số nhân (1,74 lần), muốn nhanh hơn phải thêm máy; đổi broker sang Redis chỉ bằng cấu hình, và giết worker vẫn đủ 1.000/1.000 kết quả; 1 % task chậm nhất chiếm 26 % thời gian Lean nên cần work stealing.

Sản phẩm bàn giao theo mục 4 của hướng dẫn, đối chiếu trong docs/REQUIREMENTS_MATRIX.md. **Hướng phát triển:** audit hai người và dịch ngược Lean → ngôn ngữ tự nhiên; sinh mã cho 10.000 bài thật qua API; chạy nhiều máy thật qua Redis với một REPL dự phòng đã nạp Mathlib.

PHỤ LỤC A. KHAI BÁO SỬ DỤNG AI

Học phần yêu cầu khai báo mọi phần việc có AI hỗ trợ; bản đầy đủ nằm trong docs/AI_USE_STATEMENT.md.

- **Claude** (Anthropic, mô hình cấu hình claude-opus-5-5), trợ lý lập trình và viết: viết phần lớn mã trong src/afp/, test và script thí nghiệm; chạy thí nghiệm trên máy ảo; soạn báo cáo, slide, data card và system card. Một thành viên giao việc theo hướng dẫn và slide môn học, quyết định phạm vi (ví dụ chỉ sinh mã cho 1.000 bài) và duyệt kết quả từng bước.
- **Claude Haiku 5.5**, đối tượng thí nghiệm: sinh 1.420 phát biểu Lean ứng viên; output thô lưu nguyên trong data/candidates.db.
- **Claude Sonnet và Claude Opus**, người chấm thử: chấm độc lập 100 mẫu audit để thử quy trình; không thay cho hai người chấm của nhóm.

Kiểm chứng. Lean kiểm mọi phát biểu do mô hình sinh, nhưng báo cáo không tuyên bố đúng nghĩa chỉ dựa vào compile. Ngữ nghĩa được kiểm bằng ba lớp có đo độ tin cậy. Mã có 26 test tự động; mọi bảng và hình được script sinh lại từ results/raw/. Lỗi do chính pipeline gây ra được ghi trong docs/CHANGELOG.md.

Trách nhiệm. Nếu mô hình sinh một định lý sai, nhóm, người dùng hệ thống, chịu trách nhiệm. Trước khi nộp, nhóm còn phải: chấm audit hai người; mỗi thành viên đọc và giải thích được phần mã mình phụ trách; chạy lại test và scripts/quickstart.sh trên máy của nhóm; đọc lại báo cáo và slide.

PHỤ LỤC B. PHÂN VAI VÀ ĐÓNG GÓP

Hướng dẫn (Nhóm 5, mục 5) gợi ý sáu vai trò, mỗi vai trò có người chính và người dự phòng. Nhật ký issue, pull request và commit của từng người nằm trong contributions.md.

Bảng B.1. Phân vai trong nhóm

Vai trò	Người chính	Phần trong repository
Data / evaluation	Nguyễn Việt Hưng (B25CHHT099)	ingest.py, schema.py, data/, data card
Prompt & model	Lê Quang Đức (B25CHHT086)	prompts.py, inference/, data/jobs/
Batch scheduler	Kim Anh Dũng (B25CHHT087)	queue.py, redis_backend.py, workers.py, pipeline.py
Lean worker pool	Bùi Giang Linh (B25CHHT103)	lean/, scripts/setup_lean.sh
Semantic audit	Nguyễn Mai Hạnh (B25CHHT093)	semantic.py, mutation.py, audit.py, audit/
Metrics / reproducibility	Vũ Quốc Hùng (B25CHHT098)	aggregator.py, experiments.py, study.py, runrecord.py, results/, environment/

Hai người chấm audit đề xuất: Bùi Giang Linh (rater1) và Nguyễn Mai Hạnh (rater2), chấm độc lập theo docs/AUDIT_GUIDELINE.md. Mỗi thành viên phải trả lời được về toàn bộ pipeline, không chỉ phần mình phụ trách.

PHỤ LỤC C. LỆNH TÁI LẬP

Chạy từ thư mục gốc của repository (Linux, Python 3.13, khoảng 10 GB đĩa). Ứng viên đã sinh nằm trong data/candidates.db theo TaskHash, nên các thí nghiệm không gọi lại mô hình.

```
python -m pip install -e ".[test]"   # phiên bản khóa:
environment/requirements.lock.txt
bash scripts/setup_lean.sh && export AFP_LEAN_ROOT=$HOME/lean   # Lean 4 v4.35.0,
Mathlib, REPL
bash scripts/quickstart.sh   # test, một lần chạy baseline trên T100, demo một bài (5–8
phút)
bash scripts/download_data.sh && python -m afp.ingest && python
scripts/make_manifest.py
bash scripts/run_all.sh   # mọi thí nghiệm của báo cáo, đúng thứ tự, chạy tiếp được nếu
bị ngắt
python -m afp.study all && python -m afp.plots   # bảng và hình trong results/figures/
```

Từng thí nghiệm: python -m afp.experiments <baseline|matrix|batch|scaling|retry|release|repro|fault|broker>, lưới tham số trong configs/experiments.json. Kiểm tra ngữ nghĩa: python -m afp.semantic --run q_T1K_zeroshot --gold-run gold_check. Audit người: python -m afp.audit kappa --a audit/rater1.xlsx --b audit/rater2.xlsx. Sinh lại ứng viên bằng mô hình khác: python -m afp.inference.batchfile export rồi import (xem README.md).

PHỤ LỤC D. RUN RECORD VÀ LOG

Mỗi lần chạy ghi một thư mục results/raw/runs/<run_id>/: run_record.json (mẫu Phụ lục B của hướng dẫn), config.json (tham số đầy đủ), raw_results.csv (một dòng cho mỗi ứng viên: TaskHash, trạng thái Lean, loại lỗi, thông báo của Lean, thời gian kiểm, worker, lần giao, số lần ghi), events.jsonl (worker sẵn sàng, timeout, khởi động lại REPL, worker kết thúc), memory.csv (RAM của máy và của pool REPL, CPU, mỗi 0,5 s), metrics.json và semantic.jsonl nếu có. Token và thời gian của từng request tới mô hình nằm trong results/raw/inference_jobs.json và data/jobs/. Định dạng chi tiết: docs/LOG_FORMAT.md. Ví dụ rút gọn, lần chạy baseline:


```
{"run_id": "base_T20_1req_A", "timestamp": "2026-10-26T17:32:55+0000",
 "git_commit": {"commit": "347a268", "code_dirty": false},
 "data_version": {"tier": "T20_1REQ", "tier_sha256": "a050808e…",
                  "candidates_db_sha256": "d74f7724…"},
 "machine": {"cpu": "Intel Xeon @ 2.80GHz", "vcpus": 2, "ram_gb": 7.84, "gpu": null},
 "software": {"python": "3.13.16", "lean": "4.35.0", "mathlib": "56a0058…",
              "repl": "58bdde1…"},
 "parameters": {"workers": 1, "batch": 1, "timeout_s": 10.0,
                "prompt_version": "v1_zeroshot_1req", "k": 1},
 "seed_timeout": {"tier_seed": 42, "llm_sampling_seed": null,
                  "max_heartbeats": 400000},
 "metrics": {"compile_rate": 1.0, "latency_ms_p95": 99.32, "makespan_s": 42.743},
 "status": "ok", "logs": {"raw_results.csv":
                          "results/raw/runs/base_T20_1req_A/raw_results.csv"}}
```


results/raw/runs_manifest.csv gom 98 run record thành một bảng. 45 lần chạy ngày 16/10 có trước khi dùng Git; record của chúng được ghi bù từ config.json và metrics.json, trường ghi bù liệt kê trong backfilled_fields, trường commit để trống kèm ghi chú trỏ tới c031d89 (bản chụp mã gần nhất với hôm đó). 53 lần chạy từ 26/10 ghi commit thật, không lần nào có mã chưa commit.

TÀI LIỆU THAM KHẢO

[1] Wu, Y. và cộng sự (2022). Autoformalization with Large Language Models. NeurIPS 2022.

[2] Zheng, K., Han, J. M., Polu, S. (2022). miniF2F: a cross-system benchmark for formal Olympiad-level mathematics. ICLR 2022.

[3] Azerbayev, Z. và cộng sự (2023). ProofNet: Autoformalizing and Formally Proving Undergraduate-Level Mathematics. arXiv:2302.12433.

[4] Xin, H. và cộng sự (2024). DeepSeek-Prover: Advancing Theorem Proving in LLMs through Large-Scale Synthetic Data. arXiv:2405.14333.

[5] Xin, H. và cộng sự (2024). DeepSeek-Prover-V1.5. arXiv:2408.08152. Bản Lean 4 của ProofNet và miniF2F lấy từ repository của công trình này.

[6] Ying, H. và cộng sự (2024). Lean Workbook: A large-scale Lean problem set formalized from natural language math problems. NeurIPS 2024 Datasets and Benchmarks.

[7] Li, Z. và cộng sự (2024). Autoformalize Mathematical Statements by Symbolic Equivalence and Semantic Consistency. NeurIPS 2024.

[8] Lu, J. và cộng sự (2025). FormalAlign: Automated Alignment Evaluation for Autoformalization. ICLR 2025.

[9] Hendrycks, D. và cộng sự (2021). Measuring Mathematical Problem Solving With the MATH Dataset. NeurIPS 2021 Datasets and Benchmarks. Lấy qua PRM800K math_splits (Lightman và cộng sự, 2023).

[10] Cobbe, K. và cộng sự (2021). Training Verifiers to Solve Math Word Problems (GSM8K). arXiv:2110.14168.

[11] LI, J. và cộng sự (2024). NuminaMath. Báo cáo kỹ thuật và bộ dữ liệu; không tải được trong môi trường thí nghiệm.

[12] de Moura, L., Ullrich, S. (2021). The Lean 4 Theorem Prover and Programming Language. CADE-28. Mã nguồn: Lean 4.

[13] The mathlib Community (2020). The Lean Mathematical Library. CPP 2020. Mã nguồn: Mathlib4.

[14] leanprover-community/repl: REPL của Lean 4.

[15] Cohen, J. (1960). A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1), 37–46.

[16] Yang, K. và cộng sự (2023). LeanDojo: Theorem Proving with Retrieval-Augmented Language Models. NeurIPS 2023 Datasets and Benchmarks. Tài liệu: https://leandojo.readthedocs.io/. Nguồn khởi đầu mà hướng dẫn gợi ý; nhóm không dùng, vì gọi Lean qua REPL chính thức cho phép đo trực tiếp chi phí kiểm chứng.

[17] Nguyễn Kiều Linh. Hướng dẫn bài tập lớn và slide bài giảng học phần INT4418 – Big Data Analytics (AI4Math), Học viện Công nghệ Bưu chính Viễn thông.